

HEITOR RODRIGUES SABINO

DETECÇÃO DE FRAUDES FINANCEIRAS: ALGORITMO NÃO SUPERVISIONADO DE AGRUPAMENTO HDBSCAN* EM COMPARAÇÃO COM ABORDAGENS SUPERVISIONADAS DE CLASSIFICAÇÃO RANDOM FOREST E XGBOOST

LAVRAS – MG

2026

HEITOR RODRIGUES SABINO

DETECÇÃO DE FRAUDES FINANCEIRAS: ALGORITMO NÃO SUPERVISIONADO DE AGRUPAMENTO HDBSCAN* EM COMPARAÇÃO COM ABORDAGENS SUPERVISIONADAS DE CLASSIFICAÇÃO RANDOM FOREST E XGBOOST

Trabalho de conclusão de curso apresentado à Universidade Federal de Lavras, como parte das exigências do curso de Ciências da Computação, para a obtenção do título de Bacharel.

Elaine Cecilia Gatto
Orientador

Natanael Fabrício Dacioli Batista
Coorientador

LAVRAS – MG

2026

HEITOR RODRIGUES SABINO

DETECÇÃO DE FRAUDES FINANCEIRAS: ALGORITMO NÃO SUPERVISIONADO DE AGRUPAMENTO HDBSCAN* EM COMPARAÇÃO COM ABORDAGENS SUPERVISIONADAS DE CLASSIFICAÇÃO RANDOM FOREST E XGBOOST

Trabalho de conclusão de curso apresentado à Universidade Federal de Lavras, como parte das exigências do curso de Ciências da Computação, para a obtenção do título de Bacharel.

APROVADA em - .

- UFLA
- UFLA

Elaine Cecilia Gatto
Orientador

Natanael Fabrício Dacioli Batista
Co-Orientador

LAVRAS – MG
2026

Dedico à minha mãe Cintia Maria da Silva e ao meu pai Humberto Alencar Sabino.

AGRADECIMENTOS

Agradeço aos meus orientadores, professores e amigos.

"Os circuitos de consagração social serão tanto mais eficazes quanto maior a distância social do objeto consagrado."
(Pierre Bourdie)

RESUMO

-

Palavras-chave: Fraudes Financeiras; Aprendizado de Máquina; HDBSCAN*; Agrupamento; Classificação.

ABSTRACT

-

Keywords: Financial Fraud; Machine Learning; HDBSCAN*; Clustering; Classification.

SUMÁRIO

1 Introdução 9
2 Trabalhos Correlatos 12
3 Fundamentação Teórica 16
3.1 Aprendizado de Máquina 16
3.1.1 Aprendizado Supervisionado 16
3.1.2 Aprendizado Não Supervisionado 17
3.1.3 Pipeline de Desenvolvimento e Validação 18
3.1.3.1 Estratificação 18
3.1.3.2 Principal Component Analysis (PCA) 18
3.1.3.3 Escalonamento e padronização 18
3.1.3.4 Validação Cruzada Aninhada 18
3.2 Algoritmos Supervisionados 20
3.2.1 Random Forest 20
3.2.2 Extreme Gradient Boosting 21
3.3 Algoritmo Não Supervisionado 22
3.3.1 DBSCAN* 22
3.3.2 HDBSCAN* 23
3.4 Métricas de Avaliação 24
3.4.1 Matriz de Confusão 25
3.4.2 Acurácia 25
3.4.3 Precisão 26
3.4.4 Revocação 26
3.4.5 Pontuação F1 26
3.4.6 Curva de Precisão e Revocação 26
3.4.7 Precisão Média 27
3.4.8 Índice DBCV 28
4 Metodologia 30
4.1 Conjunto de Dados 30

4.2 Pré-processamento 31
4.3 Parametrização e avaliação de consistência 36
4.3.1 Nested Cross-Validation 36
4.3.2 Grid Search 37
4.3.3 Grades de Hiperparâmetros 38
4.4 Detecção de Fraudes 40
4.4.1 Modelos Supervisionados 40
4.4.2 HDBSCAN* 40
4.4.3 Comparação dos Modelos 42
4.5 Ambiente de Execução e Bibliotecas 42
5 Resultados e Discussão 43
6 Conclusão 44
REFERÊNCIAS 45

1 INTRODUÇÃO

As fraudes financeiras abrangem atividades ilícitas de naturezas distintas. Na revisão de Ngai et al. (2011) sobre o uso da mineração de dados na detecção dessas fraudes, elas são classificadas em fraudes bancárias, fraudes de seguros, fraudes de títulos e mercadorias (*commodities*) e outras fraudes financeiras, e a fraude com cartão de crédito figura entre as fraudes bancárias. Esse tipo de fraude é uma forma de roubo de identidade em que alguém que não é o titular realiza uma transação ilícita com o cartão ou com os dados da conta, e pode partir de um cartão roubado, perdido ou falsificado (ALARFAJ et al., 2022).

A migração dos negócios para a internet e a circulação do dinheiro por meios eletrônicos, em uma economia cada vez menos dependente do dinheiro em espécie, fizeram da detecção de fraudes uma preocupação central dos sistemas bancários. Detectá-las com precisão limita as perdas diretas causadas pelas transações fraudulentas e evita que clientes legítimos sejam prejudicados pelas revisões automáticas e manuais (JURGOVSKY et al., 2018). Com o crescimento das compras online, tornou-se também mais comum a fraude sem a presença do cartão (*card-not-present*), em que apenas o número do cartão é utilizado no comércio eletrônico, e a expansão dos pagamentos digitais tem resultado em perdas anuais de bilhões de dólares (ALARFAJ et al., 2022).

Os métodos tradicionais, baseados na detecção manual, são lentos, caros e imprecisos e, diante do grande volume de dados atual, tornam-se impraticáveis, o que levou as instituições financeiras a adotar processos automatizados, apoiados em métodos estatísticos e computacionais (WEST; BHATTACHARYA, 2016). Um sistema típico de detecção combina uma ferramenta automática, que analisa cada nova transação e lhe atribui um escore de fraude, com um processo manual, no qual investigadores examinam as transações de maior escore e informam se cada uma delas é fraudulenta ou legítima (JURGOVSKY et al., 2018). As regras da ferramenta automática podem ser definidas por especialistas, a partir de cenários de fraude já conhecidos pelos investigadores, ou aprendidas dos dados por algoritmos de Aprendizado de Máquina (*Machine Learning*, ML), que reconhecem os padrões das fraudes e os procuram nas novas transações (JURGOVSKY et al., 2018).

Ngai et al. (2011) agruparam as técnicas de mineração de dados aplicadas à detecção de fraudes financeiras em seis classes: classificação, regressão, agrupamento, previsão, detecção de *outliers* e visualização. Os autores observaram que a fraude com cartão de crédito vinha recebendo grande atenção e que os modelos logísticos, as redes neurais, as redes bayesianas e as árvores de decisão eram as técnicas mais utilizadas.

A detecção de fraudes em cartões de crédito reúne, porém, dificuldades próprias. Os hábitos dos clientes e as estratégias dos fraudadores mudam ao longo do tempo, as transações legítimas superam em muito as fraudulentas e apenas uma pequena parte das transações é verificada a tempo pelos investigadores (DAL POZZOLO et al., 2018). Somam-se a isso as restrições de acesso a dados reais, uma vez que as instituições financeiras não podem expor os dados de seus clientes (SULAIMAN; SCHETININ; SANT, 2022).

Essas dificuldades pesam na escolha da abordagem. Os métodos supervisionados tratam a detecção como uma classificação binária entre fraude e transação legítima e aprendem com transações previamente rotuladas, rótulos que dependem da verificação feita pelos investigadores (JURGOVSKY et al., 2018; DAL POZZOLO et al., 2018). Os métodos de detecção de anomalias partem de outra perspectiva: modelam as transações legítimas e consideram anômalas as que não se ajustam a esse modelo, o que exige um escore capaz de quantificar o grau de anomalia de cada transação (JURGOVSKY et al., 2018). Entre esses métodos, os baseados em densidade assumem que as anomalias se encontram em regiões de baixa densidade (GHOSH et al., 2024), e o HDBSCAN* produz, a partir de uma mesma hierarquia de densidades, tanto o agrupamento dos dados quanto a detecção de *outliers* (CAMPELLO et al., 2015).

Neste contexto, o objetivo deste trabalho é avaliar o HDBSCAN*, algoritmo não supervisionado de agrupamento baseado em densidade, na detecção de fraudes em transações com cartões de crédito, comparando-o com dois classificadores supervisionados baseados em árvores de decisão, o *Random Forest* e o XGBoost. São utilizadas transações reais realizadas em setembro de 2013 por titulares de cartões europeus, das quais apenas 0,172% são fraudes (MACHINE LEARNING GROUP - UNIVERSITÉ LIBRE DE BRUXELLES, 2018).

Para isso, os objetivos específicos são preparar os dados para os três modelos, construir com o HDBSCAN* um escore de anomalia para cada transação, ajustar os hiperparâmetros dos modelos e compará-los sob o mesmo processo de avaliação. O escore do HDBSCAN* combina

a pertinência da transação ao agrupamento que lhe é atribuído e a distância aos seus vizinhos mais próximos, e as transações de maior escore são sinalizadas como suspeitas, como na ferramenta automática descrita anteriormente. Os rótulos não são usados para construir esse modelo nem para escolher os seus hiperparâmetros, selecionados pelo índice de validação de agrupamentos por densidade (*Density-Based Clustering Validation*, DBCV), que mede a qualidade do agrupamento sem utilizar rótulos (MOULAVI et al., 2014). Os hiperparâmetros do *Random Forest* e do XGBoost são escolhidos por busca em grade, e os três modelos são avaliados por validação cruzada aninhada e comparados pela área sob a curva de precisão e revocação (*Area Under the Precision-Recall Curve*, AUPRC), adequada a bases com forte desbalanceamento entre as classes (DAVIS; GOADRICH, 2006).

Parte-se da hipótese de que as fraudes ocupam regiões de baixa densidade no espaço dos atributos e podem ser reconhecidas como anomalias sem o uso de rótulos. A hipótese não é trivial, pois as fraudes podem se misturar às transações legítimas nesse espaço, já que as compras legítimas dos clientes variam muito e os fraudadores adotam estratégias difíceis de detectar (JURGOVSKY et al., 2018). Como os classificadores supervisionados aprendem diretamente com as fraudes rotuladas, eles servem de referência para medir quanto da capacidade de detecção o HDBSCAN* alcança sem essa informação, o que interessa sobretudo quando os rótulos são escassos ou chegam com atraso (DAL POZZOLO et al., 2018).

Além desta introdução, o trabalho está organizado em cinco capítulos. O Capítulo 2 apresenta os trabalhos correlatos e situa esta pesquisa em relação a estudos de detecção de fraudes em cartões de crédito. O Capítulo 3 reúne a fundamentação teórica, com os conceitos de Aprendizado de Máquina, os algoritmos comparados e as métricas de avaliação. O Capítulo 4 descreve a metodologia, com o conjunto de dados, o pré-processamento e a estratégia de validação e de ajuste dos hiperparâmetros. O Capítulo 5 apresenta e discute os resultados, e o Capítulo 6 reúne as conclusões e as sugestões de trabalhos futuros.

2 TRABALHOS CORRELATOS

Neste capítulo, será feita uma revisão de trabalhos relacionados à temática de detecção de fraudes em cartões de crédito, estabelecendo conexões e contrapontos com a pesquisa proposta neste trabalho. Em particular, serão analisadas as publicações de Alarfaj et al. (2022), Sulaiman, Schetinin e Sant (2022), Jurgovsky et al. (2018) e Ghalwash et al. (2025).

Baseando-se no mesmo *dataset* que será usado nesta pesquisa, Alarfaj et al. (2022) construíram uma revisão analisando os principais algoritmos usados na área juntamente com os desafios práticos ao analisar fraudes, tais como alta taxa de alarmes falsos, mudança constante na natureza das fraudes e o alto desbalanceamento das bases. Assim, utilizaram algoritmos tradicionais de Aprendizado de Máquina (*Machine Learning*, ML), sendo eles Árvore de Decisão, *Random Forest*, *Support Vector Machine* (SVM), Regressão Logística, *K-Nearest Neighbours* (KNN) e XGBoost, e, no Aprendizado Profundo (*Deep Learning*, DL), aplicaram e modificaram arquiteturas baseadas em Redes Neurais Convolucionais (CNN). Além disso, também propuseram balanceamento prévio dos dados, com o objetivo de minimizar a taxa de falsos negativos.

No estudo, conseguiram aprimorar os algoritmos tradicionais até certo ponto, porém conseguiram resultados superiores com CNN, afirmando que a adição de camadas ocultas na arquitetura da rede neural contribuiu para isso. Com essa abordagem, o modelo proposto alcançou 99,9% de acurácia, 93% de precisão, 85,71% na pontuação F1 e 98% de área sob a curva ROC (AUC). O destaque de acurácia geral foi de 99,72%, obtido ao combinar um modelo base com uma CNN de 20 camadas.

Enquanto o trabalho de Alarfaj et al. (2022) foca, em sua maioria, em abordagens supervisionadas, este trabalho busca analisar a eficiência, especificamente, de um algoritmo não supervisionado baseado em densidade para a identificação de fraudes, assumindo como hipótese que fraudes são anomalias (*outliers*).

Em uma revisão da literatura sobre o tema, Sulaiman, Schetinin e Sant (2022) compararam estudos que aplicaram técnicas de ML à detecção de fraudes em cartões de crédito, considerando também a confidencialidade dos dados dos clientes. Foram analisadas abordagens baseadas em *Random Forest*, Redes Neurais Artificiais (RNA), SVM e KNN, além de métodos

híbridos e de preservação de privacidade. Os autores apontam como principais desafios da área a diversidade das transações fraudulentas, o desbalanceamento das bases e a dificuldade de obter dados reais, uma vez que as instituições financeiras não podem expor os dados de seus clientes em razão da legislação europeia de proteção de dados (*General Data Protection Regulation*, GDPR). Em relação ao desbalanceamento, relatam que a sobreamostragem sintética da classe minoritária (*Synthetic Minority Oversampling Technique*, SMOTE) melhora o desempenho dos algoritmos, mas pode introduzir ruído e sobreposição entre as classes, levando ao sobreajuste.

Na comparação, os autores constataram que o aprendizado supervisionado é a prática predominante, com destaque para SVM, KNN, *Naive Bayes*, Regressão Logística e Árvore de Decisão, e que as abordagens híbridas superam o uso de um único classificador. O desempenho dos estudos revisados é reportado, em sua maioria, pela acurácia. Também são apontadas limitações de alguns algoritmos em cenários reais, como a lentidão do *Random Forest* no treinamento e na predição e a queda de desempenho do KNN diante de grandes volumes de dados. Como solução, os autores propõem um modelo que combina uma RNA com aprendizado federado (*Federated Learning*), em que cada instituição treina o modelo localmente e somente os modelos treinados, e não os dados, são combinados em um servidor central. A proposta é descrita em etapas, sem avaliação experimental no artigo.

Assim como em Alarfaj et al. (2022), a revisão de Sulaiman, Schetinin e Sant (2022) evidencia o predomínio de abordagens supervisionadas, que dependem de transações previamente rotuladas para o treinamento. Neste trabalho, o HDBSCAN* não utiliza os rótulos para construir o modelo nem para escolher os seus hiperparâmetros, que são selecionados pelo índice DBCV, uma medida da qualidade do agrupamento que dispensa rótulos (MOULAVI et al., 2014). Os rótulos servem apenas para a avaliação, e o HDBSCAN* é comparado com dois classificadores supervisionados, o *Random Forest* e o XGBoost. Além disso, a acurácia é pouco informativa em bases com desbalanceamento extremo como a deste trabalho, em que as fraudes representam 0,172% das transações, pois um modelo que classificasse todas as transações como legítimas já alcançaria 99,8% de acurácia. Por isso, os modelos são avaliados por métricas baseadas na precisão, como discutido a seguir.

Jurgovsky et al. (2018) trataram a detecção de fraudes como uma tarefa de classificação de sequências, partindo da ideia de que o histórico de compras do titular ajuda a reconhecer uma

transação fraudulenta. Para isso, empregaram redes neurais recorrentes do tipo *Long Short-Term Memory* (LSTM), que recebem as transações anteriores de cada cartão, e as compararam com o *Random Forest*, que avalia cada transação isoladamente e foi escolhido como referência por ter se mostrado um modelo de bom desempenho na tarefa e ser amplamente utilizado na área. Os experimentos usaram transações reais de cartões de crédito registradas entre março e maio de 2015 e fornecidas pela Worldline, empresa que também colaborou na coleta do conjunto de dados deste trabalho. As compras foram separadas em online e presenciais, e, no período de teste, as fraudes representavam 0,42% e 0,05% das transações, respectivamente. Os autores ainda testaram três conjuntos de atributos, o mais completo deles com variáveis que resumem a atividade do titular nas 24 horas anteriores. Para a avaliação, adotaram a área sob a curva de precisão e revocação (*Area Under the Precision-Recall Curve*, AUPRC), justificando que a curva ROC é pouco sensível ao aumento de falsos positivos quando os exemplos negativos superam em muito os positivos, o que é indesejável na detecção de fraudes.

Nas compras presenciais, a LSTM elevou a AUPRC média de 0,138, obtida pelo *Random Forest*, para 0,200 com os atributos originais. Com as variáveis agregadas, porém, os dois modelos se equipararam, com 0,241 e 0,246, respectivamente. Nas compras online, os resultados dos dois modelos foram praticamente iguais, e as variáveis agregadas elevaram a AUPRC de cerca de 0,18 para valores entre 0,38 e 0,40. Por fim, ao comparar as fraudes corretamente identificadas, os autores observaram que a LSTM e o *Random Forest* detectam fraudes diferentes, o que sugere a combinação das duas abordagens.

Assim como Jurgovsky et al. (2018), este trabalho compara a abordagem estudada com o *Random Forest* e avalia os modelos pela AUPRC, métrica adequada a bases com desbalanceamento extremo. Por outro lado, o conjunto de dados utilizado aqui não identifica o titular de cada transação, o que impede a construção de sequências por cartão e de variáveis agregadas por titular. Assim, em vez de explorar o histórico de cada cliente, este trabalho investiga se as fraudes podem ser reconhecidas como pontos de baixa densidade no espaço dos atributos, por meio do HDBSCAN*.

Mais próximo da abordagem deste trabalho, Ghalwash et al. (2025) empregaram o algoritmo de agrupamento baseado em densidade DBSCAN na detecção de fraudes em cartões de crédito. Os autores utilizaram transações de titulares europeus de 2023, disponibilizadas no

Kaggle e também com atributos transformados por PCA, e montaram três subconjuntos com 40.400, 80.800 e 121.200 transações, todos com 1% de fraudes. O DBSCAN foi aplicado de forma iterativa, e os rótulos de agrupamento gerados foram adicionados à base como novos atributos. Em seguida, *Random Forest*, KNN e SVM foram treinados sobre essa base enriquecida e combinados por votação disjuntiva, em que uma transação é considerada fraudulenta se ao menos um dos classificadores a apontar como tal, estratégia que prioriza a revocação.

O modelo proposto alcançou revocação de até 99,5%, pontuação F1 de até 99,8% e acurácia e precisão de 100% nos três subconjuntos, superando os classificadores isolados e a votação por maioria. Os próprios autores, no entanto, reconhecem que, quando as fraudes representam menos de 0,2% dos registros, o DBSCAN pode não formar agrupamentos significativos de fraudes, já que as transações minoritárias tendem a aparecer como ruído ou *outliers*, e que a escolha dos seus parâmetros nesse cenário não é trivial.

Enquanto Ghalwash et al. (2025) utilizam o agrupamento por densidade como etapa auxiliar de classificadores supervisionados, este trabalho emprega o HDBSCAN* como o próprio detector. A limitação apontada pelos autores corresponde justamente à hipótese explorada aqui: em uma base com 0,172% de fraudes, espera-se que as transações fraudulentas ocupem regiões de baixa densidade e se destaquem como anomalias. Além disso, o HDBSCAN* dispensa o raio de vizinhança fixo do DBSCAN (CAMPELLO et al., 2015), um dos parâmetros cuja escolha os autores apontam como difícil.

3 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os conceitos teóricos fundamentais que embasam o desenvolvimento deste trabalho.

3.1 Aprendizado de Máquina

O Aprendizado de Máquina (*Machine Learning*, ML) consiste no desenvolvimento de algoritmos que capacitam computadores a aprender a realizar tarefas a partir de dados, sem a necessidade de programação explícita para cada regra de execução, segundo Samuel (1959), que introduziu o tema.

Segundo Rezazadeh (2025), as principais abordagens do Aprendizado de Máquina dividem-se em quatro categorias: o aprendizado supervisionado, baseado em dados rotulados; o aprendizado não supervisionado, aplicado a dados não rotulados; o aprendizado semissupervisionado, que combina as duas abordagens anteriores ao utilizar simultaneamente dados rotulados e não rotulados; e o aprendizado por reforço, no qual o agente aprende por tentativa e erro, a partir de recompensas e penalidades obtidas na interação com o ambiente. As subseções seguintes tratam do aprendizado supervisionado e do não supervisionado, que fundamentam os modelos comparados neste trabalho.

3.1.1 Aprendizado Supervisionado

O aprendizado supervisionado destaca-se como uma das metodologias mais proeminentes dentro do Aprendizado de Máquina (BZDOK; KRZYWINSKI; ALTMAN, 2018). Essa abordagem é caracterizada por sua capacidade de aprender com experiências passadas mediante a utilização de conjuntos de dados históricos previamente rotulados. Ao analisar esses dados rotulados durante a fase de treinamento, o algoritmo adquire a capacidade de prever ou atribuir os rótulos apropriados a novos dados não classificados.

As técnicas primárias empregadas no aprendizado supervisionado são a classificação e a regressão, as quais diferem fundamentalmente na natureza da previsão alvo (BZDOK; KRZYWINSKI; ALTMAN, 2018):

- **Classificação:** É aplicada quando o vetor de saída previsto é de natureza categórica ou discreta. A função primária de um classificador é atribuir novos pontos de dados a categorias distinhas predefinidas, utilizando o conhecimento extraído da base de treinamento (GOPAL, 2018).

- **Regressão:** Ao contrário da classificação, a técnica de regressão é utilizada quando a saída desejada é de natureza numérica ou contínua, conforme contextualizado por Rezazadeh (2025).

3.1.2 Aprendizado Não Supervisionado

A segunda principal técnica em ML é o aprendizado não supervisionado, empregado na construção de modelos preditivos em situações onde não há disponibilidade de dados previamente rotulados para o treinamento (SOLORIO-FERNÁNDEZ; CARRASCO-OCHOA; MARTÍNEZ-TRINIDAD, 2020). Consequentemente, não há informações prévias sobre a correlação exata entre as características de entrada e saída.

Nesse caso, o papel do algoritmo é permitir que os dados falem por si, revelando as estruturas intrínsecas do conjunto sem a orientação de rótulos predefinidos, o que caracteriza o aprendizado não supervisionado como um processo orientado pelos dados (REZAZADEH, 2025). Uma das distinções mais significativas em relação à metodologia supervisionada é que as abordagens não supervisionadas frequentemente requerem um grau mais elevado de interpretação humana. Uma vez que não existem métricas estatísticas simples que definam um resultado de forma binária (como um rótulo "certo ou errado"), o processo baseia-se fortemente em medidas de estatística descritiva e técnicas de visualização de dados (GOPAL, 2018). As principais categorias do aprendizado não supervisionado incluem:

- **Agrupamento (Clustering):** Considerada a técnica não supervisionada mais prevalente (KONONENKO; KUKAR, 2007), o agrupamento visa organizar os dados com base em suas semelhanças intrínsecas. O objetivo é garantir que os itens dentro de um mesmo agrupamento (*cluster*) compartilhem características fortes (como formato, cor ou dimensão), enquanto exibem alta dissimilaridade em relação aos itens de outros agrupamentos (IGLESIAS; ZSEBY; ZIMEK, 2021; SHALEV-SHWARTZ; BEN-DAVID, 2014).

- **Associação:** Esta técnica foca na descoberta de dependências ou correlações entre entradas individuais de dados para, posteriormente, mapeá-las em conjunto (DIKE et al., 2018).

- **Redução de Dimensionalidade:** Uma técnica adjacente que visa simplificar a complexidade dos dados não rotulados, extraindo as características mais relevantes sem perder a representatividade da informação (GOPAL, 2018).

3.1.3 Pipeline de Desenvolvimento e Validação

A implementação de modelos de aprendizado supervisionado e não supervisionado seguem um fluxo metodológico geral, onde percorre etapas de pré-processamento até a avaliação final.

3.1.3.1 Estratificação

3.1.3.2 Principal Component Analysis (PCA)

A Análise de Componentes Principais é um procedimento de estatística multivariável que reduz a dimensionalidade do *dataset*, isso de forma a permitir de maneira intuitiva e estruturada a interpretação das principais *features* inerentes a um conjunto de dados multivariado complexo (GREENACRE et al., 2022).

3.1.3.3 Escalonamento e padronização

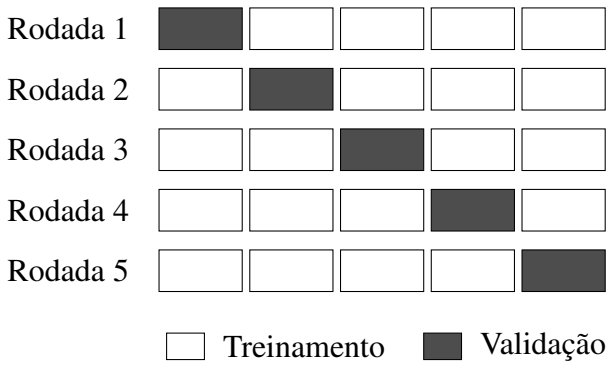
3.1.3.4 Validação Cruzada Aninhada

Com o fim de mitigar o viés de otimismo na estimativa de desempenho, assegurar a estabilidade estatística, garantindo a consistência e confiabilidade dos resultados, e, também, fazer a cobertura integral do conjunto de dados durante o processo de modelagem, adotou-se a metodologia de validação cruzada aninhada (*nested cross-validation*). Para Krstajic et al. (2014), parafraseando a literatura clássica (STONE, 1974; VARMA; SIMON, 2006), essa abordagem é indispensável quando se realiza otimização de hiperparâmetros, pois impede o

vazamento de informações (*data leakage*) ao separar de forma clara a etapa de calibração da etapa de generalização.

Segundo Berrar (2024), a validação cruzada *k-fold* consiste em particionar o conjunto de dados disponível, por amostragem aleatória sem reposição, em k subconjuntos disjuntos (*folds*) de tamanho aproximadamente igual. Em cada uma de k rodadas, o modelo é treinado com $k-1$ subconjuntos, que formam o conjunto de treinamento, e avaliado no subconjunto restante, denominado conjunto de validação. O procedimento se repete até que cada subconjunto tenha sido utilizado uma vez para validação, e a média das k medidas de desempenho constitui a estimativa obtida por validação cruzada. Quando a amostragem é estratificada, cada subconjunto preserva a proporção das classes do conjunto original. A Figura 3.1 ilustra o procedimento para $k = 5$.

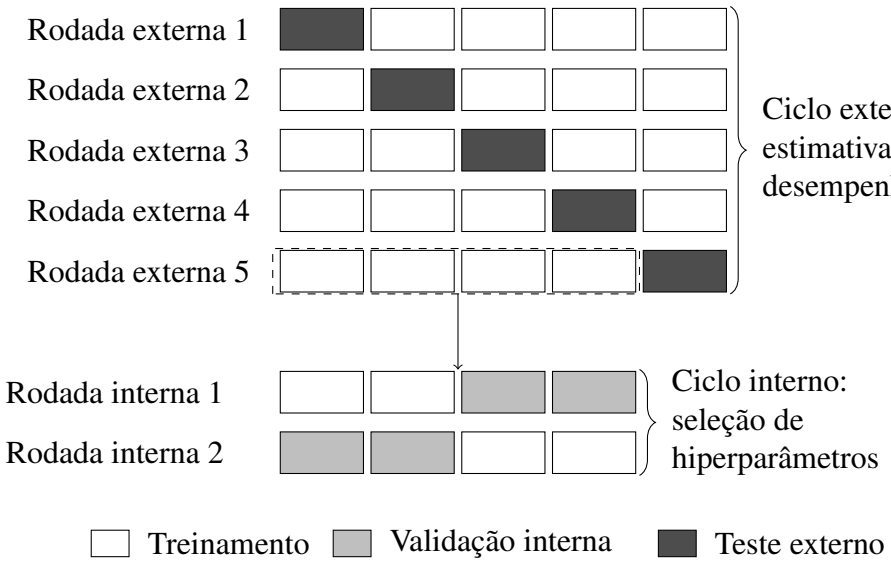
Figura 3.1 – Validação cruzada *k-fold* com $k = 5$.



Fonte: Adaptado de Berrar (2024).

A validação cruzada aninhada executa esse procedimento em dois níveis. O ciclo externo corresponde a uma validação cruzada *k-fold* convencional, destinada a estimar o desempenho de generalização. Em cada rodada externa, o conjunto de treinamento é submetido a uma nova validação cruzada, o ciclo interno, na qual são avaliadas as configurações candidatas de hiperparâmetros. A configuração de melhor desempenho médio no ciclo interno é utilizada para treinar o modelo com todo o conjunto de treinamento externo, que é então avaliado no subconjunto de teste externo. A estimativa final corresponde à média dos desempenhos obtidos nas k rodadas externas, e as configurações selecionadas podem diferir entre as rodadas. Desse modo, a seleção de hiperparâmetros permanece separada da avaliação do modelo (BERRAR, 2024). A Figura 3.2 ilustra o esquema.

Figura 3.2 – Validação cruzada aninhada com cinco rodadas externas e duas internas. O ciclo interno é representado apenas para a quinta rodada externa.



Fonte: Adaptado de Berrar (2024).

Quanto à seleção de hiperparâmetros, cada ciclo interno indica a configuração de melhor desempenho, e, dentre as configurações assim obtidas, seleciona-se a de melhor desempenho, que é aplicada a todas as rodadas externas. O desempenho é medido pela AUPRC nos modelos supervisionados (Seção 3.4.7) e, no HDBSCAN*, pelo DBCV, índice que avalia a qualidade do agrupamento sem utilizar rótulos (Seção 3.4.8).

3.2 Algoritmos Supervisionados

Neste tópico, serão descritos os algoritmos supervisionados baseados em árvores de decisão selecionados para o embasamento comparativo do estudo.

3.2.1 Random Forest

Uma árvore de decisão é a menor unidade formadora de um modelo *Random Forest* (RF). Assim, segundo Zhang e Singer (2010), a estrutura de uma árvore de decisão é constituída por camadas, onde a quantidade total dos dados a serem analisados se encontra na raiz, e assim as estruturas subsequentes possuem menor quantidade de dados. À medida que os da-

dos atravessam a árvore, eles passam por nós internos até serem classificados em subgrupos terminais mutuamente exclusivos, as folhas (WEST; BHATTACHARYA, 2016).

O modelo RF é construído a partir de várias árvores de decisão, com o fim de superar a instabilidade das estimativas de uma árvore isolada (BIAU; SCORNET, 2016). Nesse sentido, Zhang e Singer (2010) afirmam que, isoladamente, cada árvore não representa um bom modelo, mas a combinação delas gera ganho de valor. Assim, por ser uma junção de modelos (árvores de decisão), o *Random Forest* é categorizado como modelo de *ensemble* (SALMAN; KALAKECH; STEITI, 2024).

O método utiliza a técnica *Bagging* (*Bootstrap Aggregating*). Esse processo consiste em gerar múltiplos conjuntos de treinamento a partir da base de dados original, selecionando as amostras de forma aleatória e com reposição. Cada subconjunto treina uma árvore independente. O resultado final do modelo é obtido pela agregação das respostas de todos os chamados *base learners* (modelos menores como árvores de decisão): utiliza-se a média aritmética para tarefas de regressão ou o voto da maioria para classificação (SALMAN; KALAKECH; STEITI, 2024). Além da amostragem com reposição, a cada divisão de nó o RF considera apenas um subconjunto de variáveis sorteado entre as disponíveis (BIAU; SCORNET, 2016), o que torna as árvores mais diferentes e menos correlacionadas entre si e confere maior estabilidade à agregação (PROBST; WRIGHT; BOULESTEIX, 2019). A variância da floresta corresponde ao produto da correlação entre as árvores pela variância de uma árvore individual; por isso, a combinação de árvores pouco correlacionadas reduz a variância do modelo, que diminui à medida que o número de árvores aumenta, sem conduzir ao sobreajuste (BIAU; SCORNET, 2016).

3.2.2 Extreme Gradient Boosting

O *Extreme Gradient Boosting*, comumente abreviado como XGBoost, consiste em uma otimização avançada da arquitetura *Gradient Boosting Machine* (GBM). Assim como o seu predecessor, o XGBoost é classificado como um modelo *ensemble*, pois utiliza uma coleção de modelos preditivos individuais (como as árvores de decisão) integrados em uma estrutura única para atingir um objetivo comum.

Como o GBM atua como o alicerce teórico do XGBoost, é fundamental compreender a sua mecânica. O modelo é composto por um conjunto de preditores individuais, denominados

base learners (WADE, 2020). Diferentemente do *Random Forest*, em que a construção das árvores ocorre de maneira estritamente independente, o diferencial do *Gradient Boosting* é o seu aprendizado sequencial fundamentado na correção de erros. O GBM otimiza seus resultados construindo novas árvores direcionadas especificamente para corrigir as falhas mapeadas nas iterações anteriores, adicionando-as ao conjunto para complementar as estruturas já existentes (WADE, 2020; PAN, 2018).

A partir dessa base conceitual, o XGBoost surge como uma evolução de alta performance. O seu principal diferencial técnico é a exigência rigorosa para a expansão do modelo: uma nova ramificação só é construída se o ganho de eficiência gerado por essa inclusão for estritamente superior ao ganho da estrutura anterior. Consequentemente, caso o benefício da inclusão seja inferior ao da árvore que a originou, o algoritmo realiza uma poda, interrompendo o crescimento daquele nó (CHEN; GUESTRIN, 2016).

3.3 Algoritmo Não Supervisionado

3.3.1 DBSCAN*

O DBSCAN* é uma variante do DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) (ESTER et al., 1996; CAMPELLO et al., 2015) e, como ele, é um algoritmo de agrupamento que identifica padrões nos dados fundamentado na estimativa de níveis mínimos de densidade (SCHUBERT et al., 2017). O funcionamento do modelo depende da definição de dois parâmetros principais: um raio de alcance (ε) e um número mínimo de vizinhos (*minPts*). Durante a análise, um objeto é classificado como um "ponto central"(*core point*) se a quantidade de vizinhos presentes dentro do seu raio ε for igual ou superior ao limiar estabelecido por *minPts* (SCHUBERT et al., 2017; CAMPELLO et al., 2015).

A premissa fundamental do algoritmo é localizar regiões contínuas que satisfaçam essa densidade mínima, isolando-as de áreas mais esparsas do conjunto de dados (SCHUBERT et al., 2017). No DBSCAN*, os agrupamentos (*clusters*) são formados apenas por pontos centrais: dois pontos centrais contidos no raio ε um do outro são diretamente alcançáveis e pertencem ao mesmo agrupamento, que se expande de forma transitiva para englobar os demais pontos

centrais alcançáveis a partir dos seus membros. Os pontos que não são centrais são rotulados como ruído (*noise*) (CAMPELLO et al., 2015).

3.3.2 HDBSCAN*

O HDBSCAN* (*Hierarchical DBSCAN**) é um algoritmo hierárquico de agrupamento baseado em densidade que aprimora o método de agrupamento por densidade, o DBSCAN* (CAMPELLO et al., 2015). Ao contrário do DBSCAN*, que exige a definição prévia do raio ε, o HDBSCAN* tem como parâmetro principal o número mínimo de pontos, m_{pts}, equivalente ao m_{pts} do DBSCAN* e tratado como um fator de suavização das estimativas de densidade (CAMPELLO et al., 2015).

A principal diferença do HDBSCAN* em relação ao DBSCAN* está no tratamento do raio. Em vez de fixar um valor de ε, o HDBSCAN* calcula, para cada ponto x_p, a distância núcleo $d_{core}(x_p)$, definida como a distância de x_p ao seu m_{pts}-ésimo vizinho mais próximo, contando o próprio x_p. A distância núcleo é, portanto, o menor raio ε para o qual x_p satisfaz a condição de ponto central (CAMPELLO et al., 2015), e é intuitivo que ela cresça à medida que a vizinhança do ponto se torna mais esparsa. A partir dela, define-se a distância de alcançabilidade mútua entre dois pontos x_p e x_q (Equação 3.1), que corresponde ao menor raio ε para o qual os dois pontos são alcançáveis um a partir do outro (CAMPELLO et al., 2015):

$$d_{mreach}(x_p, x_q) = \max\{d_{core}(x_p), d_{core}(x_q), d(x_p, x_q)\} \tag{3.1}$$

em que $d(x_p, x_q)$ é a distância original entre os dois pontos.

Essas distâncias definem o grafo de alcançabilidade mútua, um grafo completo cujos vértices são os pontos do conjunto de dados e cujas arestas têm como peso a distância de alcançabilidade mútua entre eles (CAMPELLO et al., 2015). Esse grafo é conceitual e não precisa ser materializado, pois os pesos das arestas são calculados sob demanda para a obtenção da sua árvore geradora mínima (NETO et al., 2022). As arestas dessa árvore são então removidas em ordem decrescente de peso, o que equivale a reduzir progressivamente o raio ε: a cada remoção, os componentes conectados de pontos centrais formam os agrupamentos, e os demais pontos passam a ser ruído (CAMPELLO et al., 2015). Dessa forma, enquanto o DBSCAN* produz

um único agrupamento para o raio escolhido, o HDBSCAN* produz uma hierarquia que reúne todos os agrupamentos que o DBSCAN* obteria para o valor de m_{pts} e para qualquer $\varepsilon \in [0, \infty)$ (CAMPELLO et al., 2015).

Para obter uma partição única, os agrupamentos são extraídos da hierarquia por um critério de estabilidade, segundo o qual os agrupamentos mais proeminentes tendem a persistir por uma faixa maior de densidades (CAMPELLO et al., 2013; CAMPELLO et al., 2015). Nesse processo, componentes com menos pontos que um tamanho mínimo de agrupamento, m_{clSize}, são considerados espúrios e rotulados como ruído (CAMPELLO et al., 2015).

O valor de m_{pts} continua a influenciar o resultado. Valores maiores exigem mais vizinhos para que um ponto seja central, o que aumenta as distâncias núcleo e tende a formar agrupamentos mais suaves e possivelmente maiores, enquanto valores menores permitem agrupamentos mais detalhados e menores (GHOSH et al., 2024). Assim, diferentes faixas de m_{pts} podem levar a resultados distintos (NETO et al., 2021; NETO et al., 2022), o que torna necessária a busca por esse parâmetro.

Na detecção não supervisionada de anomalias, parte-se da premissa de que as anomalias se encontram em regiões de baixa densidade do conjunto de referência (GHOSH et al., 2024). Um método global clássico é o dos k vizinhos mais próximos (*k-Nearest Neighbors*, kNN), em que o escore de anomalia de um ponto é a distância ao seu k-ésimo vizinho mais próximo (RAMASWAMY; RASTOGI; SHIM, 2000), grandeza análoga à distância núcleo do HDBSCAN* (CAMPELLO et al., 2015).

Neste estudo, as fraudes não são identificadas apenas como os pontos rotulados como ruído. Cada transação recebe um escore de anomalia que combina a sua pertinência ao agrupamento atribuído pelo HDBSCAN* e a distância ao seu m_{pts}-ésimo vizinho mais próximo, e as transações de maior escore são sinalizadas como suspeitas.

3.4 Métricas de Avaliação

Para quantificar o desempenho dos modelos de classificação, utilizam-se métricas que comparam a classe atribuída a cada exemplo com a sua classe real. A escolha dessas métricas depende do objetivo da modelagem e da estrutura dos dados, e, na detecção de fraudes, o forte

desbalanceamento entre as classes torna algumas delas pouco informativas. Além dessas métricas, a seção apresenta o índice DBCV, que avalia a qualidade de um agrupamento sem utilizar a classe real.

3.4.1 Matriz de Confusão

Em uma classificação binária, cada exemplo pertence a uma de duas classes, a positiva ou a negativa, e o classificador atribui a ele uma delas. Neste trabalho, a classe positiva corresponde à fraude. A matriz de confusão cruza a classe predita com a classe real e organiza as quatro combinações possíveis. Os verdadeiros positivos (VP) são as fraudes classificadas como fraude, e os falsos negativos (FN), as fraudes classificadas como legítimas. Os verdadeiros negativos (VN) são as transações legítimas classificadas como legítimas, e os falsos positivos (FP), as transações legítimas classificadas como fraude (FAWCETT, 2006). O Quadro 3.1 apresenta a disposição dessas contagens.

Quadro 3.1 – Matriz de confusão.

		Classe real	
		Fraude	Legítima
Classe predita	Fraude	VP	FP
	Legítima	FN	VN

Fonte: Adaptado de Fawcett (2006).

3.4.2 Acurácia

A acurácia é a proporção de exemplos classificados corretamente (Equação 3.2) (FAWCETT, 2006).

$$\text{Acurácia} = \frac{VP+VN}{VP+VN+FP+FN} \tag{3.2}$$

Em bases desbalanceadas, porém, a acurácia não é uma medida adequada (CHAWLA et al., 2002). Como a classe negativa domina as contagens, um classificador que atribui todos os exemplos à classe majoritária alcança acurácia elevada sem identificar nenhum exemplo positivo.

3.4.3 Precisão

A precisão é a proporção de verdadeiros positivos entre os exemplos classificados como positivos (Equação 3.3) (FAWCETT, 2006). Na detecção de fraudes, ela indica quantas das transações sinalizadas como suspeitas são de fato fraudes.

$$\text{Precisão} = \frac{VP}{VP + FP} \tag{3.3}$$

3.4.4 Revocação

A revocação (*recall*), também chamada de sensibilidade ou taxa de verdadeiros positivos, é a proporção dos exemplos positivos que o classificador identifica (Equação 3.4) (FAWCETT, 2006). Na detecção de fraudes, ela indica quantas das fraudes existentes são sinalizadas.

$$\text{Revocação} = \frac{VP}{VP + FN} \tag{3.4}$$

3.4.5 Pontuação F1

A pontuação F1 (*F-measure*) combina a precisão e a revocação em um único valor por meio da média harmônica entre as duas (Equação 3.5) (FAWCETT, 2006). Por se tratar de uma média harmônica, o valor resultante é baixo sempre que uma das duas métricas é baixa.

$$F1 = 2 \cdot \frac{\text{Precisão} \cdot \text{Revocação}}{\text{Precisão} + \text{Revocação}} \tag{3.5}$$

3.4.6 Curva de Precisão e Revocação

As métricas anteriores dependem de um limiar de decisão. Os modelos utilizados neste trabalho atribuem a cada exemplo um escore, como a probabilidade estimada de fraude ou o grau de anomalia, e o exemplo é classificado como positivo quando esse escore supera o limiar (FAWCETT, 2006). Cada limiar produz um par de valores de precisão e revocação, e a curva de precisão e revocação reúne esses pares ao longo de todos os limiares possíveis (DAVIS; GOADRICH, 2006).

Em problemas com grande desbalanceamento entre as classes, a curva ROC (*Receiver Operating Characteristic*), que relaciona as taxas de verdadeiros e de falsos positivos, pode apresentar uma visão excessivamente otimista do desempenho, e a curva de precisão e revocação é apontada como alternativa (DAVIS; GOADRICH, 2006). Com muitos exemplos negativos, um aumento expressivo no número de falsos positivos altera pouco a taxa de falsos positivos, mas reduz a precisão, que compara os falsos positivos aos verdadeiros positivos (DAVIS; GOADRICH, 2006).

3.4.7 Precisão Média

A área sob a curva de precisão e revocação (AUPRC) resume a curva em um único valor, entre 0 e 1. O cálculo dessa área exige cuidado, pois a interpolação linear entre os pontos da curva de precisão e revocação é incorreta e superestima o desempenho (DAVIS; GOADRICH, 2006).

Os métodos de detecção de anomalias produzem uma ordenação completa dos exemplos pelo grau de anomalia, e uma das medidas usadas para avaliar essa ordenação é a precisão média (*Average Precision*, AP) (CAMPOS et al., 2016). Com os exemplos ordenados de forma decrescente pelo escore, seja $P@n$ a precisão (Equação 3.3) calculada entre as n primeiras posições. A precisão média é a média dos valores de $P@n$ nas posições ocupadas pelos exemplos positivos (Equação 3.6) (ZHANG; ZHANG, 2009):

$$AP = \frac{1}{|O|} \sum_{o \in O} P@\text{pos}(o) \tag{3.6}$$

em que O é o conjunto de exemplos positivos, $|O|$ é o seu tamanho e $\text{pos}(o)$ é a posição do exemplo o na ordenação. Como a revocação aumenta na mesma proporção a cada exemplo positivo encontrado na ordenação, a Equação 3.6 equivale a somar, em cada um desses pontos, a precisão multiplicada pelo aumento da revocação. A precisão média corresponde, portanto, à área sob a curva de precisão e revocação calculada em degraus, sem a interpolação linear entre os pontos.

Avaliar a precisão nas posições dos exemplos positivos é adequado para bases desbalanceadas (CAMPOS et al., 2016). A precisão média vale 1 quando todos os exemplos positivos

ocupam as primeiras posições, e o seu valor esperado para uma ordenação aleatória é a proporção de exemplos positivos na base (CAMPOS et al., 2016). Como depende apenas da ordenação dos escores, ela avalia o modelo sem a escolha de um limiar de decisão.

3.4.8 Índice DBCV

As métricas anteriores comparam as predições com a classe real e dependem, portanto, de exemplos rotulados. A qualidade de um agrupamento, por sua vez, pode ser avaliada por critérios internos, que utilizam apenas os próprios dados (MOULAVI et al., 2014). A maior parte desses critérios relaciona a dispersão dentro dos grupos à separação entre eles e foi concebida para grupos de formato convexo, de modo que falha diante de grupos de formato arbitrário e não é definida para pontos de ruído (MOULAVI et al., 2014).

Para os agrupamentos baseados em densidade, Moulavi et al. (2014) propuseram o índice de validação de agrupamentos por densidade (*Density-Based Clustering Validation*, DBCV). O índice parte da ideia de que, em um bom agrupamento por densidade, a região menos densa de cada grupo ainda é mais densa que a região mais densa ao seu redor. Para medir essas densidades, cada objeto recebe uma distância núcleo calculada em relação a todos os demais objetos do seu grupo, interpretada como o inverso da sua densidade, e essa distância ocupa o lugar da distância núcleo na distância de alcançabilidade mútua (Equação 3.1). Com essas distâncias, constrói-se a árvore geradora mínima de cada grupo, da qual se extraem duas grandezas, consideradas apenas as arestas e os objetos internos, isto é, os que não estão nas extremidades da árvore. A esparsidade de densidade de um grupo C_i, $DSC(C_i)$, é o maior peso entre as arestas internas da sua árvore e corresponde à sua região menos densa. A separação de densidade entre dois grupos C_i e C_j, $DSPC(C_i, C_j)$, é a menor distância de alcançabilidade mútua entre os objetos internos das suas árvores e corresponde à região mais densa entre eles (MOULAVI et al., 2014).

A validade de cada grupo compara essas duas grandezas (Equação 3.7) (MOULAVI et al., 2014):

$$V_C(C_i) = \frac{\min\limits_{j \neq i} DSPC(C_i, C_j) - DSC(C_i)}{\max\left(\min\limits_{j \neq i} DSPC(C_i, C_j), DSC(C_i)\right)} \tag{3.7}$$

O valor é positivo quando o grupo é mais denso em seu interior do que na região que o separa dos demais, e negativo no caso contrário. O DBCV é a média dessas validades ponderada pelo tamanho de cada grupo (Equação 3.8) (MOULAVI et al., 2014):

$$DBCV(C) = \sum_{i=1}^{l} \frac{|C_i|}{|O|} V_C(C_i) \tag{3.8}$$

em que C é o agrupamento avaliado, l é o seu número de grupos e $|O|$ é o número total de objetos, incluindo os rotulados como ruído. O índice varia entre -1 e 1, e valores maiores indicam agrupamentos melhores (MOULAVI et al., 2014). Como o denominador inclui o ruído, que não contribui com nenhuma parcela da soma, rotular objetos como ruído reduz o índice, o que penaliza as soluções que deixam grande parte dos dados fora dos grupos (MOULAVI et al., 2014).

Por dispensar a classe real, o DBCV permite avaliar e selecionar algoritmos de agrupamento por densidade e os seus parâmetros (MOULAVI et al., 2014). Neste trabalho, ele orienta a escolha dos hiperparâmetros do HDBSCAN*, em busca de agrupamentos bem definidos e bem separados do ruído, região em que se espera encontrar as fraudes.

4 METODOLOGIA

Este capítulo descreve os procedimentos adotados no desenvolvimento do trabalho: o conjunto de dados e o seu pré-processamento, a estratégia de validação e de ajuste dos hiperparâmetros, a forma como cada modelo detecta as fraudes e o ambiente de execução. Todas as etapas aleatórias, como a divisão dos *folds* e o treinamento dos modelos supervisionados, usam a mesma semente para garantir reprodutibilidade.

4.1 Conjunto de Dados

Os dados referentes a transações financeiras reais são sigilosos em virtude da natureza comprometedora do dado e por ser de domínio privado, visto isso, pesquisas acerca de investigações de crimes financeiros enfrentam tal barreira. Nesse sentido, o conjunto de dados utilizado possui informações limitadas acerca das colunas com o objetivo de sanar a questão da confidencialidade dos registros.

Assim, o conjunto de dados manipulado neste trabalho foi disponibilizado pelo Machine Learning Group da Université Libre de Bruxelles, ao qual está acessível pelo site Kaggle (MACHINE LEARNING GROUP - UNIVERSITÉ LIBRE DE BRUXELLES, 2018).

O *dataset* contém transações em cartões de crédito em setembro de 2013, feitas por titulares de cartões europeus e foram coletadas e analisadas durante uma pesquisa em colaboração com a Worldline, uma empresa francesa de serviços de pagamento e transações, fundada em 1974. Foram capturadas 284.807 transações ao longo de um período de dois dias, onde apenas 492 são classificadas como fraudulentas. Logo, o conjunto de dados é altamente desbalanceado considerando a elevada desproporção entre as classes, com a classe positiva (fraude) representando 0,172% de todas as transações.

Em relação às *features*, o *dataset* possui 31 variáveis numéricas das quais 28 (V1, V2, ..., V28) delas foram submetidas ao processo de Análise de Componente Principal (PCA), uma técnica padrão e amplamente utilizada para reduzir a dimensionalidade de tais conjuntos de dados, assim aumentando a interpretabilidade e, de forma paralela, minimizando a perda de informação (ABDI; WILLIAMS, 2010). Isso acontece por meio da criação de novas variáveis não correlacionadas que maximizam sucessivamente a variância. Concomitantemente, o PCA

colabora para a anonimização dos dados (ALARFAJ et al., 2022). As *features* que não foram submetidas ao PCA são *amount*, *class* e *time*.

Tabela 4.1 – Descrição das colunas do conjunto de dados.

Variável	**Descrição**	**Tipo**
Time	Número de segundos decorridos entre a transação atual e a primeira transação registrada no conjunto de dados.	Contínua
V1 a V28	Componentes principais resultantes de uma transformação de redução de dimensionalidade (PCA). Os dados originais não são fornecidos para proteger a identidade dos usuários e informações sensíveis.	Contínua
Amount	Valor monetário da transação.	Contínua
Class	Variável alvo (rótulo). Assume o valor 1 caso a transação seja uma fraude e 0 caso seja uma transação genuína.	Binária

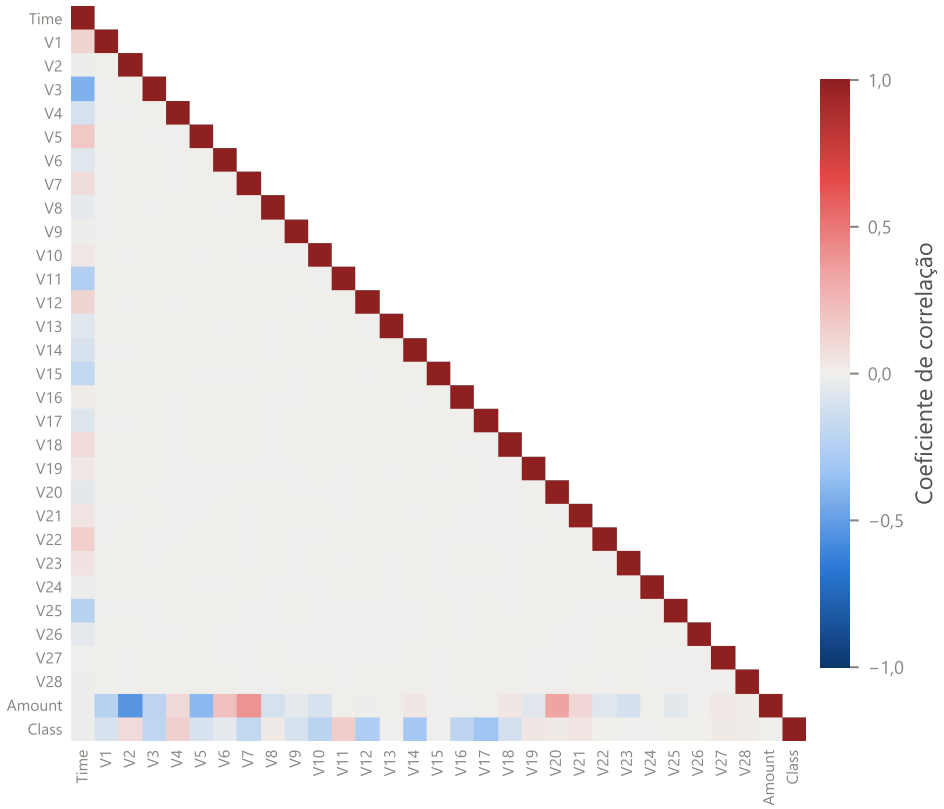
4.2 Pré-processamento

Através de processos de análise exploratória acerca da base, notou-se que não há necessidade de tratamentos quanto a dados faltantes ou nulos. O conjunto de dados foi originalmente validado nesse sentido. A base contém, no entanto, 1.081 transações que repetem outra em todas as variáveis. Elas são retiradas do conjunto de treinamento do HDBSCAN*, pois cópias de um mesmo ponto ficam a distância zero umas das outras e aumentariam artificialmente a densidade estimada na sua região; os conjuntos de validação e de teste permanecem completos, para que a avaliação cubra todas as transações.

A Figura 4.1 mostra a correlação de Pearson entre as variáveis: as componentes V1 a V28 não são correlacionadas entre si, por construção do PCA (ABDI; WILLIAMS, 2010), e as correlações restantes envolvem Time e Amount. Por conta da anonimização advinda do PCA, não é possível interpretar o que essas correlações representam, e nenhuma variável foi retirada ou fundida por causa delas: todas são mantidas nos modelos com suas devidas transformações e padronizações.

A variável Time registra os segundos decorridos desde a primeira transação da base e cresce ao longo dos dois dias de coleta, de modo que, em seu valor bruto, apenas ordena as

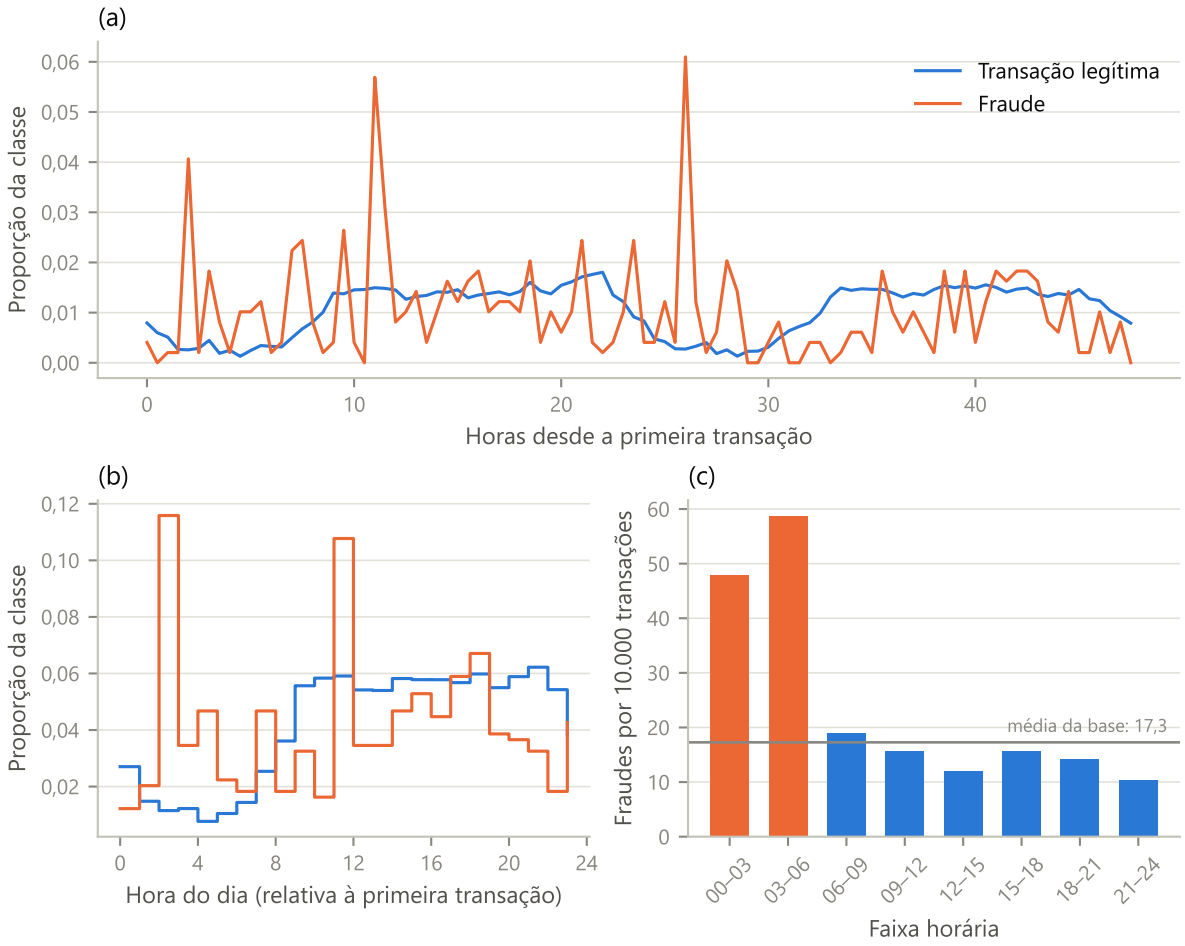
Figura 4.1 – Correlação de Pearson entre as variáveis da base.



Fonte: Do autor (2026).

transações, como um identificador. A Figura 4.2 compara o horário das transações legítimas e das fraudulentas: convertidas em hora do dia, as fraudes se concentram em determinadas faixas de horário (painel b), e a faixa de maior risco tem 5,6 vezes mais fraudes por transação que a de menor risco (painel c). Por isso, o valor bruto de Time não é usado por nenhum dos modelos. Dele se extrai apenas a posição da transação no ciclo de 24 horas, cuja distribuição difere entre fraudes e transações legítimas, como confirma o teste de Kolmogorov-Smirnov sobre a hora do dia ($D = 0{,}197$; $p < 0{,}001$). O valor de D é a maior diferença entre as proporções acumuladas das duas classes ao longo do dia: em certo horário, a fração das fraudes ocorridas até ali difere em 19,7 pontos percentuais da fração das transações legítimas. O valor de p indica que uma diferença desse tamanho seria muito improvável se as duas classes tivessem a mesma distribuição de horários.

Figura 4.2 – Horário das transações legítimas e fraudulentas. (a) Distribuição ao longo da coleta; (b) distribuição por hora do dia; (c) fraudes por 10.000 transações em cada faixa de três horas.



Fonte: Do autor (2026).

Como a base não informa o horário da primeira transação, a hora do dia é contada a partir desse instante, e não do horário de fato. O deslocamento é o mesmo para todas as transações, de modo que a duração do ciclo, a distância entre os horários e a diferença entre fraudes e transações legítimas se preservam; apenas os rótulos das faixas da Figura 4.2 deixam de corresponder ao horário local.

A coluna Time passou então pela transformação para representar a hora do dia. O HDBSCAN*, o *Random Forest* e o XGBoost recebem essa mesma representação.

Durante a etapa de pré-processamento foram feitas algumas transformações quanto aos dados. Assim, foram feitas padronizações nas variáveis exceto pela variável Class e pelas que

passaram pelo processo de PCA, pois pressupõe-se que em ordem de aplicar a transformação PCA, é necessário que passe pelo processo de padronização antes, visto que o método é sensível às escalas.

Já quanto à padronização das variáveis Amount e Time (após a transformação), foram escalonadas, pois, em ML, esse processo transforma os valores das variáveis de acordo com uma regra definida, assim essas *features* escalonadas têm o mesmo grau de influência e esse método é imune à escolha das unidades (HUANG; LI; XIE, 2015).

Para a hora do dia e para a coluna Amount foi feito o escalonamento padrão, *Z-score standardization* (Equação 4.1):

$$z = \frac{x - \mu_X}{\sigma_X} \tag{4.1}$$

em que μ_X e σ_X são a média e o desvio padrão da variável em questão, calculados apenas sobre o conjunto de treinamento de cada *fold* e aplicados, com os mesmos valores, aos conjuntos de validação e de teste. O HDBSCAN*, o *Random Forest* e o XGBoost recebem as variáveis padronizadas da mesma forma.

Antes do escalonamento, a variável Amount foi transformada pelo logaritmo (Equação 4.2), uma vez que dados financeiros apresentam alta assimetria e presença severa de *outliers* (valores transacionais atípicos):

$$x' = \ln(1 + x) \tag{4.2}$$

em que x é o valor da transação. A soma de 1 mantém a transformação definida quando o valor é zero.

A assimetria foi medida pelo coeficiente tradicional de assimetria amostral (Coeficiente de Assimetria de Fisher, ou Coeficiente de Momento de Assimetria), g_1 (JOANES; GILL, 1998) (Equação 4.3):

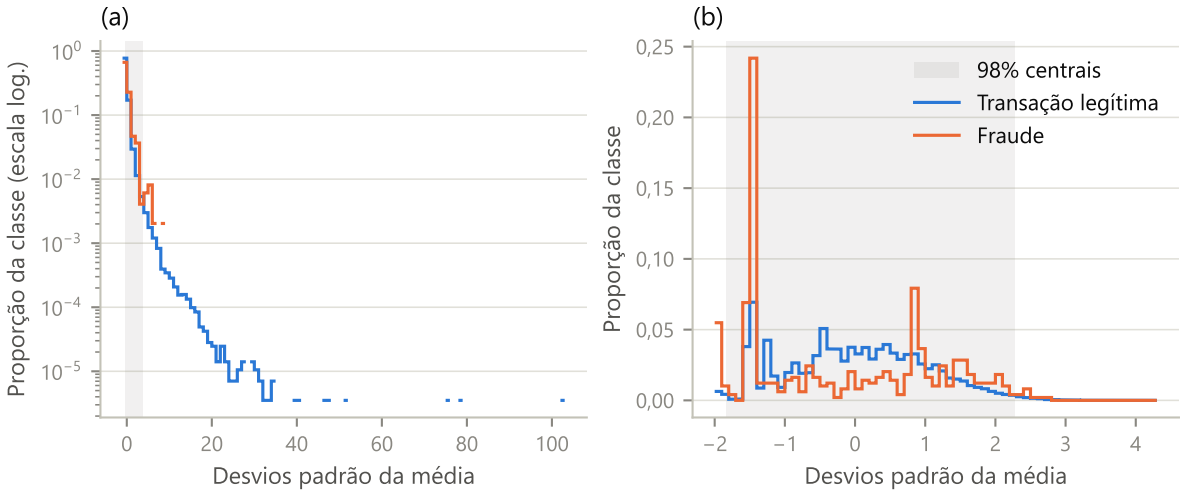
$$g_1 = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^3}{\left(\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2\right)^{3/2}} \tag{4.3}$$

em que n é o número de transações e $\bar{x}$ é a média dos valores. O coeficiente é nulo para uma distribuição simétrica e positivo quando há uma cauda longa à direita. Em Amount, ele vale 16,98, pois a mediana é de 22,00, mas o maior valor chega a 25.691,16. Após o logaritmo, o coeficiente cai para 0,16.

Sem o logaritmo, a cauda inflaria o desvio padrão usado no escalonamento. Assim, 98% das transações ficariam entre $-0{,}35$ e 3,72 desvios padrão da média, enquanto a maior delas ficaria a 102. Como a distância euclidiana soma as diferenças ao quadrado de cada variável, Amount sozinha decidiria os vizinhos das transações extremas e quase não diferenciaria as demais, o que contraria o objetivo do escalonamento, citado anteriormente, de dar às variáveis o mesmo grau de influência. Com o logaritmo, 98% das transações ficam entre $-1{,}83$ e 2,28 desvios padrão, e a mais extrema, a 4,2. A Figura 4.3 compara as duas distribuições.

O logaritmo preserva a ordem dos valores, de modo que a maior transação continua a mais distante da média. O que muda é a medida da diferença, que passa a ser relativa, de modo que, exemplificando, ir de 10 a 100 equivale a ir de 1.000 a 10.000. Além disso, os valores extremos da base não são fraudes. Nenhuma das 284 transações acima de 3.000 é fraudulenta, e a maior fraude é de 2.125,87. As consequências disso para os modelos são discutidas no Capítulo 5.

Figura 4.3 – Distribuição de Amount padronizada pelo Z-score, (a) sem e (b) com o logaritmo. A faixa cinza contém 98% das transações. Na figura, a média e o desvio padrão são os da base completa, e nos modelos vêm do conjunto de treinamento de cada fold.



Fonte: Do autor (2026).

4.3 Parametrização e avaliação de consistência

Um passo importante no que tange à parametrização e validação da consistência do modelo é a estratégia a ser tomada para tal feito. Visto isso, visamos realizar validação cruzada aninhada juntamente com a busca em grade como estratégia.

4.3.1 Nested Cross-Validation

O procedimento operacional se estrutura a partir de dois ciclos de repetição interdependentes: Ciclo externo (*Outer Loop*) e Ciclo Interno (*Inner Loop*).

No externo, o conjunto de dados global é particionado/replicado de maneira pseudo-aleatória em k subconjuntos (são denominados *folds*) mutuamente exclusivos e de dimensões aproximadamente equivalentes. Tanto a replicação quanto as divisões em particionamento de treino e teste são realizadas de forma estratificada, assegurando que cada *fold* preserve a proporção original da variável alvo (classes lícita e fraudulenta). De acordo com Kohavi (1995), a amostragem estratificada é superior ao esquema regular, pois reduz de forma consistente tanto o viés quanto a variância das estimativas de desempenho em conjuntos de dados do mundo real.

Já no interno, os dados de treinamento provenientes do ciclo externo são novamente subdivididos em novos subconjuntos, treino e validação. É dentro deste ciclo secundário que se executa a busca em grade (*Grid Search*) para a calibração dos hiperparâmetros do algoritmo. O modelo é treinado e validado em partições internas isoladas, mapeando-se a configuração paramétrica que otimiza a métrica alvo.

Sumariamente, a relevância fundamental da validação cruzada aninhada está no fato de que as instâncias usadas na escolha dos parâmetros ideais, o ciclo interno, jamais coincidem com os dados empregados para inferir a capacidade real preditiva do modelo, o ciclo externo. Desse modo, o método assegura a cobertura da base, analisando todas as transações de dados de forma independente, assim resultando em uma estimativa de erro metodologicamente mais robusta, imparcial e menos propensa ao *overfitting*.

Para detalhar a distribuição dos dados, este trabalho adotou uma divisão em 5 *folds* externos, compostos por subconjuntos de treino e teste. Essa partição seguiu a proporção de 80% dos dados originais para o treinamento e 20% para o teste. Em valores absolutos, cada *fold* contém

aproximadamente 227.845 instâncias para treino e 56.961 para teste. Adicionalmente, a partir de cada conjunto de treinamento dos *folds* externos, realizou-se uma nova subdivisão interna para validação. Como resultado, o conjunto de treinamento final de cada *fold* passou a contar com cerca de 182.276 instâncias, enquanto o conjunto de validação reuniu 45.569 instâncias. Conforme mencionado previamente, todas as partições foram geradas de forma estratificada, garantindo a manutenção da mesma proporção de fraudes presente na base original.

4.3.2 Grid Search

Para que um algoritmo de aprendizado de máquina extraia o máximo de sua capacidade preditiva e se adapte à topologia intrínseca do conjunto de dados, é imperativo que seus hiperparâmetros sejam calibrados. Conforme evidenciado pela literatura recente (SHAMS et al., 2023), a utilização de configurações padrão ou arbitrárias frequentemente resulta em desempenho subótimo, tornando a etapa de otimização paramétrica um requisito fundamental para a robustez do modelo.

Neste trabalho, adotou-se a Busca em Grade (*Grid Search*) como mecanismo de otimização dentro do ciclo interno da validação aninhada. O *Grid Search* é um método de varredura exaustiva que constrói uma malha multidimensional contendo todos os valores predefinidos para os parâmetros do algoritmo. Durante a fase de calibração, o modelo é instanciado, treinado e avaliado iterativamente para cada combinação única presente nesta grade (SHAMS et al., 2023).

De acordo com estudos comparativos em otimização de hiperparâmetros (LIASHCHYNSKYI; LIASHCHYNSKYI, 2019), o *Grid Search* demonstra-se a abordagem mais adequada e rigorosa para espaços de busca de baixa e média dimensionalidade. Embora possua um custo computacional linearmente proporcional ao tamanho da grade, o método assegura a exploração completa do domínio de parâmetros estipulados, garantindo a localização da combinação que matematicamente maximiza a métrica de desempenho na partição de validação, sem o risco de convergência prematura para ótimos locais.

Nos três modelos, a busca em grade é executada no ciclo interno da validação cruzada aninhada, e a seleção segue o procedimento descrito na Seção 3.1.3.4. Em cada uma das cinco rodadas, as combinações da grade são treinadas no treino interno e avaliadas na validação in-

terna, e a de melhor desempenho é indicada. Dentre as cinco combinações indicadas, a de melhor desempenho é selecionada e aplicada a todas as rodadas do ciclo externo, nas quais o modelo é treinado com todo o treino externo e avaliado no teste externo. Assim, em cada rodada, o modelo avaliado é treinado sem as transações do teste. O desempenho é medido pela AUPRC no *Random Forest* e no XGBoost e pelo DBCV no HDBSCAN*.

No HDBSCAN*, o DBCV (Seção 3.4.8) é calculado sobre o agrupamento obtido no treino interno, sem os rótulos, de modo que nenhuma informação sobre as fraudes participa da escolha. O índice parte da matriz de distâncias entre todos os pares de objetos (MOULAVI et al., 2014). Devido a isso, tem-se uma complexidade quadrática de espaço e de tempo. Assim, o DBCV é calculado sobre uma amostra do conjunto de dados em questão, estratificada pelos grupos encontrados, incluindo o ruído.

4.3.3 Grades de Hiperparâmetros

As grades de hiperparâmetros foram definidas a partir das recomendações da literatura para cada algoritmo, de modo a explorar valores em torno das configurações usuais sem tornar a busca inviável.

No *Random Forest*, a grade abrange o número de variáveis sorteadas em cada divisão, o tamanho da amostra usada por árvore, o tamanho mínimo das folhas, o número de árvores e a ponderação das classes. Segundo Probst, Wright e Boulesteix (2019), os três primeiros controlam o compromisso entre a baixa correlação das árvores e a qualidade de cada uma. O número de variáveis sorteadas é um dos hiperparâmetros centrais do algoritmo: valores menores geram árvores mais diferentes e menos correlacionadas, que se agregam com mais estabilidade, mas cada árvore tende a ser pior, por escolher as divisões entre poucas variáveis candidatas (PROBST; WRIGHT; BOULESTEIX, 2019). A grade parte do valor padrão para classificação, a raiz quadrada do número de variáveis (PROBST; WRIGHT; BOULESTEIX, 2019), e inclui frações maiores, que favorecem árvores individualmente melhores. O tamanho da amostra tem efeito semelhante, pois amostras menores produzem árvores mais diversas e reduzem o tempo de execução, mas cada árvore é treinada com menos transações (PROBST; WRIGHT; BOULESTEIX, 2019). Por isso, a grade compara a amostra completa com frações menores do treino. O tamanho mínimo das folhas tem como padrão o valor 1 na classificação, e valores

maiores reduzem o tempo de execução em bases grandes (PROBST; WRIGHT; BOULESTEIX, 2019), como a deste trabalho. Quanto ao número de árvores, um número maior tende a produzir predições mais precisas e não leva ao sobreajuste (BIAU; SCORNET, 2016), mas o tempo de execução cresce linearmente com ele (PROBST; WRIGHT; BOULESTEIX, 2019), o que limita a grade a poucas centenas de árvores. Por fim, como as fraudes são raras, a grade compara o treinamento sem ponderação com a ponderação das classes, que atribui à classe minoritária um peso maior no treinamento das árvores e cujos pesos são um parâmetro de ajuste essencial do modelo (CHEN; LIAW; BREIMAN, 2004).

No XGBoost, a grade abrange o número de árvores, a profundidade máxima de cada árvore, a taxa de aprendizado e o parâmetro γ. A taxa de aprendizado (*shrinkage*) multiplica por um fator a contribuição de cada nova árvore, o que reduz a influência de cada árvore individual e deixa espaço para que as árvores seguintes aperfeiçoem o modelo (CHEN; GUESTRIN, 2016). Por isso, ela é combinada na grade com diferentes números de árvores. O parâmetro γ penaliza o número de folhas na função objetivo regularizada do XGBoost, cujo termo de regularização suaviza os pesos aprendidos e evita o sobreajuste (CHEN; GUESTRIN, 2016). Com γ nulo, nenhuma penalidade é aplicada, e valores positivos tornam as árvores mais simples. A profundidade e a taxa de aprendizado abrangem a configuração que Chen e Guestrin (2016) adotam como comum em seus experimentos, com profundidade máxima 8 e taxa de aprendizado 0,1. Para o desbalanceamento, as fraudes recebem no treinamento um peso igual à razão entre o número de transações legítimas e o de fraudes no treino, valor indicado pela documentação da biblioteca para classes desbalanceadas (XGBOOST DEVELOPERS, 2026).

No HDBSCAN*, a grade varia o número mínimo de pontos, m_{pts}, que atua como fator de suavização das estimativas de densidade (Seção 3.3.2) e equivale ao número mínimo de vizinhos do DBSCAN*. Para o DBSCAN, recomenda-se que esse número seja o dobro da dimensão dos dados (SANDER et al., 1998 apud SCHUBERT et al., 2017), e valores maiores podem melhorar os resultados em bases com muito ruído, muito grandes ou de alta dimensão (SCHUBERT et al., 2017). Com as 31 variáveis deste trabalho, a recomendação corresponde a 62 vizinhos, e, como a base é grande, com cerca de 182 mil transações em cada treino interno, a grade explora valores de m_{pts} acima desse ponto de partida. O tamanho mínimo de agrupamento, m_{clSize}, atua

como um controle adicional e independente da suavização (CAMPELLO et al., 2015), e o seu valor também foi definido pela busca em grade.

4.4 Detecção de Fraudes

Os três modelos atribuem a cada transação um escore que cresce com a suspeita de fraude, e a transação é classificada como fraudulenta quando o escore alcança um limiar (FAWCETT, 2006). O escore e o limiar, porém, são obtidos de formas diferentes nos modelos supervisionados e no HDBSCAN*.

4.4.1 Modelos Supervisionados

O *Random Forest* e o XGBoost são treinados com as transações rotuladas do treino e estimam, para cada transação, a probabilidade de fraude, que serve de escore. No *Random Forest*, essa probabilidade é a média das probabilidades estimadas pelas árvores, e a de cada árvore é a fração de transações da classe na folha em que a transação é classificada (SCIKIT-LEARN DEVELOPERS, 2026). No XGBoost, treinado com o objetivo de regressão logística para classificação binária, a saída do modelo é a própria probabilidade (XGBOOST DEVELOPERS, 2026). A transação é classificada como fraude quando a probabilidade estimada de fraude supera a de transação legítima (SCIKIT-LEARN DEVELOPERS, 2026). Como as fraudes são raras, os dois modelos compensam o desbalanceamento durante o treinamento pela ponderação das classes (Seção 4.3.3).

4.4.2 HDBSCAN*

O HDBSCAN* é ajustado sobre as transações do treino, sem os rótulos e sem as duplicatas (Seção 4.2), com a distância euclidiana entre as variáveis padronizadas. Cada transação recebe um grau de anomalia construído a partir de dois escores. O primeiro é o complemento da probabilidade de pertinência, que mede a intensidade com que a transação pertence ao agrupamento atribuído (JAIN; MURTY; FLYNN, 1999 apud OYEWOLE; THOPIL, 2023). Essa probabilidade é proporcional ao quanto a transação persiste como parte do agrupamento e é nula para as transações rotuladas como ruído (NVIDIA, 2026). Para as transações da validação

e do teste, que não participam do ajuste, a pertinência é estimada pela atribuição aproximada de cada uma ao agrupamento obtido no treino, sem reagrupar os dados (NVIDIA, 2026). O segundo escore é a distância da transação ao seu m_{pts}-ésimo vizinho mais próximo entre as transações do treino, sem contar a própria transação, que corresponde ao escore de anomalia kNN (RAMASWAMY; RASTOGI; SHIM, 2000) e é análoga à distância núcleo do HDBSCAN* (Seção 3.3.2).

Os dois escores têm naturezas e escalas distintas, uma probabilidade e uma distância. Para combiná-los, cada escore é convertido no seu posto percentil em relação aos escores do treino, isto é, na fração das transações do treino com escore menor ou igual (Equação 4.4):

$$F_j(s) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\left[s_{ij} \leq s\right] \tag{4.4}$$

em que s_{ij} é o j-ésimo escore da i-ésima transação do treino, n é o número de transações do treino e $\mathbf{1}[\cdot]$ vale 1 quando a condição entre colchetes é verdadeira e 0 caso contrário. O grau de anomalia de uma transação x é a média dos postos dos dois escores (Equação 4.5):

$$a(x) = \frac{1}{2}\left[F_1\big(1 - p(x)\big) + F_2\big(d_{m_{pts}}(x)\big)\right] \tag{4.5}$$

em que $p(x)$ é a probabilidade de pertinência de x e $d_{m_{pts}}(x)$ é a distância de x ao seu m_{pts}-ésimo vizinho no treino. O grau de anomalia varia entre 0 e 1 e só se aproxima de 1 quando a transação é extrema, em relação ao treino, nos dois escores. Como os postos usam apenas a distribuição do treino, nenhuma estatística da validação ou do teste participa do cálculo. O rótulo de ruído, por sua vez, não é usado diretamente como regra de decisão, mas entra no grau de anomalia por meio da pertinência, que é nula para o ruído.

A transação é sinalizada como fraude quando o seu grau de anomalia alcança o limiar τ, o quantil de ordem $1 - \alpha$ dos graus de anomalia das transações do treino (Equação 4.6):

$$\tau = Q_{1-\alpha}\left(\{a(x_i)\}_{i=1}^{n}\right) \tag{4.6}$$

em que $Q_{1-\alpha}$ é o quantil de ordem $1 - \alpha$, x_i é a i-ésima transação do treino e α é a fração de alertas. Desse modo, cerca de uma fração α das transações é sinalizada, o que reproduz a

restrição dos sistemas reais de detecção, em que os investigadores conseguem verificar apenas poucos alertas por dia e as empresas se preocupam sobretudo com a precisão dos alertas gerados (DAL POZZOLO et al., 2018). Neste trabalho, a fração de alertas foi fixada em 0,1% das transações, da mesma ordem de grandeza da proporção de fraudes na base, de 0,172%.

4.4.3 Comparação dos Modelos

A comparação entre os modelos usa a AUPRC, calculada pela precisão média (Seção 3.4.7) sobre o escore contínuo de cada modelo, a probabilidade de fraude nos supervisionados e o grau de anomalia no HDBSCAN*. Como depende apenas da ordenação das transações, a AUPRC não é afetada pelos limiares, que diferem entre os modelos. A precisão, a revocação e a pontuação F1 são calculadas sobre as classes atribuídas pela regra de decisão de cada modelo e complementam a análise, mas dependem do limiar adotado. Todas as métricas são calculadas no teste de cada rodada externa, e o desempenho de cada modelo é a média entre as cinco rodadas.

4.5 Ambiente de Execução e Bibliotecas

O desenvolvimento foi realizado na linguagem Python, com a biblioteca scikit-learn (PEDREGOSA et al., 2011), que fornece o *Random Forest*, o pré-processamento e as métricas de avaliação, a biblioteca XGBoost (CHEN; GUESTRIN, 2016) e a biblioteca cuML, que oferece, em unidades de processamento gráfico (*Graphics Processing Units*, GPU), implementações de algoritmos de aprendizado de máquina equivalentes às do scikit-learn (RASCHKA; PATTERSON; NOLET, 2020) e foi usada no HDBSCAN* e na busca de vizinhos mais próximos. Os experimentos dos três modelos foram executados no ambiente Google Colab, em uma máquina com uma GPU NVIDIA L4, de 22,5 GiB de memória, 53 GiB de memória RAM e 12 processadores virtuais.

5 RESULTADOS E DISCUSSÃO

6 CONCLUSÃO

REFERÊNCIAS

ABDI, H.; WILLIAMS, L. J. Principal component analysis. **Wiley Interdiscip. Rev. Comput. Stat.**, Wiley, v. 2, n. 4, p. 433–459, jul. 2010.

ALARFAJ, F. K. et al. Credit card fraud detection using state-of-the-art machine learning and deep learning algorithms. **IEEE Access**, Institute of Electrical and Electronics Engineers (IEEE), v. 10, p. 39700–39715, 2022.

BERRAR, D. **Cross-validation**. 2024. *Preprint* submetido à Encyclopedia of Bioinformatics and Computational Biology, 2. ed., Elsevier.

BIAU, G.; SCORNET, E. A random forest guided tour. **TEST**, v. 25, n. 2, p. 197–227, jun. 2016.

BZDOK, D.; KRZYWINSKI, M.; ALTMAN, N. Machine learning: supervised methods. **Nature Methods**, v. 15, n. 1, p. 5–6, 2018. ISSN 1548-7105. Disponível em: <https://doi.org/10.1038/nmeth.4551>.

CAMPELLO, R. J. G. B. et al. A framework for semi-supervised and unsupervised optimal extraction of clusters from hierarchies. **Data Mining and Knowledge Discovery**, v. 27, n. 3, p. 344–371, 2013.

CAMPELLO, R. J. G. B. et al. Hierarchical density estimates for data clustering, visualization, and outlier detection. **ACM Transactions on Knowledge Discovery from Data**, v. 10, n. 1, p. 1–51, jul. 2015.

CAMPOS, G. O. et al. On the evaluation of unsupervised outlier detection: measures, datasets, and an empirical study. **Data Mining and Knowledge Discovery**, v. 30, n. 4, p. 891–927, 2016.

CHAWLA, N. V. et al. SMOTE: Synthetic minority over-sampling technique. **Journal of Artificial Intelligence Research**, v. 16, p. 321–357, 2002.

CHEN, C.; LIAW, A.; BREIMAN, L. **Using Random Forest to Learn Imbalanced Data**. [S.l.], 2004.

CHEN, T.; GUESTRIN, C. XGBoost: A scalable tree boosting system. **arXiv preprint arXiv:1603.02754**, mar. 2016.

DAL POZZOLO, A. et al. Credit card fraud detection: A realistic modeling and a novel learning strategy. **IEEE Transactions on Neural Networks and Learning Systems**, v. 29, n. 8, p. 3784–3797, 2018.

DAVIS, J.; GOADRICH, M. The relationship between precision-recall and ROC curves. In: **Proceedings of the 23rd International Conference on Machine Learning**. [S.l.]: ACM, 2006. p. 233–240.

DIKE, H. U. et al. Unsupervised learning based on artificial neural network: A review. In: **2018 IEEE International Conference on Cyborg and Bionic Systems (CBS)**. [S.l.: s.n.], 2018. p. 322–327.

ESTER, M. et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In: **Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)**. [S.l.: s.n.], 1996. p. 226–231.

FAWCETT, T. An introduction to ROC analysis. **Pattern Recognition Letters**, v. 27, n. 8, p. 861–874, 2006.

GHALWASH, M. A. et al. Enhancing credit card fraud detection using DBSCAN-augmented disjunctive voting ensemble. **Scientific Reports**, Springer Nature, v. 15, p. 39754, 2025.

GHOSH, K. et al. **Unsupervised Parameter-free Outlier Detection using HDBSCAN* Outlier Profiles**. 2024. *Preprint* arXiv:2411.08867.

GOPAL, M. **Applied Machine Learning**. McGraw Hill Education (India) Private Limited, 2018. ISBN 9789353160265. Disponível em: <https://books.google.com.br/books?id=BsKlDwAAQBAJ>.

GREENACRE, M. et al. Principal component analysis. **Nat. Rev. Methods Primers**, Springer Science and Business Media LLC, v. 2, n. 1, dez. 2022.

HUANG, J.; LI, Y.-F.; XIE, M. An empirical analysis of data preprocessing for machine learning-based software cost estimation. **Inf. Softw. Technol.**, Elsevier BV, v. 67, p. 108–127, nov. 2015.

IGLESIAS, F.; ZSEBY, T.; ZIMEK, A. Clustering refinement. **International Journal of Data Science and Analytics**, Springer, v. 12, n. 4, p. 333–353, 2021.

JAIN, A. K.; MURTY, M. N.; FLYNN, P. J. Data clustering: A review. **ACM Computing Surveys**, v. 31, n. 3, p. 264–323, 1999.

JOANES, D. N.; GILL, C. A. Comparing measures of sample skewness and kurtosis. **Journal of the Royal Statistical Society: Series D (The Statistician)**, v. 47, n. 1, p. 183–189, 1998.

JURGOVSKY, J. et al. Sequence classification for credit-card fraud detection. **Expert Systems with Applications**, Elsevier, v. 100, p. 234–245, 2018.

KOHAVI, R. A study of cross-validation and bootstrap for accuracy estimation and model selection. In: **Proceedings of the 14th International Joint Conference on Artificial Intelligence - Volume 2**. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1995. (IJCAI'95), p. 1137–1143.

KONONENKO, I.; KUKAR, M. **Machine Learning and Data Mining**. [S.l.]: Elsevier, 2007.

KRSTAJIC, D. et al. Cross-validation pitfalls when selecting and assessing regression and classification models. **J. Cheminform.**, Springer Science and Business Media LLC, v. 6, n. 1, p. 10, mar. 2014.

LIASHCHYNSKYI, P.; LIASHCHYNSKYI, P. Grid search, random search, genetic algorithm: A big comparison for nas. **arXiv preprint arXiv:1912.06059**, dez. 2019.

MACHINE LEARNING GROUP - UNIVERSITÉ LIBRE DE BRUXELLES. **Credit Card Fraud Detection**. Kaggle, 2018. Disponível em: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>. Acesso em: 02 jun. 2026.

MOULAVI, D. et al. Density-based clustering validation. In: **Proceedings of the 2014 SIAM International Conference on Data Mining**. [S.l.]: SIAM, 2014. p. 839–847.

NETO, A. C. A. et al. CORE-SG: Efficient computation of multiple MSTs for density-based methods. In: **Proceedings of the 38th IEEE International Conference on Data Engineering (ICDE)**. [S.l.: s.n.], 2022. p. 951–964.

NETO, A. C. A. et al. Efficient computation and visualization of multiple density-based clustering hierarchies. **IEEE Transactions on Knowledge and Data Engineering**, v. 33, n. 8, p. 3075–3089, ago. 2021.

NGAI, E. W. T. et al. The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature. **Decision Support Systems**, v. 50, n. 3, p. 559–569, 2011.

NVIDIA. **cuML API Reference**. 2026. Documentação da biblioteca cuML, entradas HDBSCAN e approximate_predict. Disponível em: <https://docs.nvidia.com/cuml/latest/api/>. Acesso em: 23 out. 2026.

OYEWOLE, G. J.; THOPIL, G. A. Data clustering: Application and trends. **Artificial Intelligence Review**, v. 56, p. 6439–6475, 2023.

PAN, B. Application of XGBoost algorithm in hourly PM2.5 concentration prediction. **IOP Conf. Ser. Earth Environ. Sci.**, IOP Publishing, v. 113, p. 012127, fev. 2018.

PEDREGOSA, F. et al. Scikit-learn: Machine learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825–2830, 2011.

PROBST, P.; WRIGHT, M. N.; BOULESTEIX, A.-L. Hyperparameters and tuning strategies for random forest. **WIREs Data Mining and Knowledge Discovery**, p. e1301, 2019.

RAMASWAMY, S.; RASTOGI, R.; SHIM, K. Efficient algorithms for mining outliers from large data sets. In: **Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data**. [S.l.: s.n.], 2000. p. 427–438.

RASCHKA, S.; PATTERSON, J.; NOLET, C. Machine learning in Python: Main developments and technology trends in data science, machine learning, and artificial intelligence. **Information**, v. 11, n. 4, p. 193, 2020.

REZAZADEH, S. Review of machine learning. **TMP Universal Journal of Research and Review Archives**, v. 4, n. 2s, Jun 2025. Disponível em: <https://tmp.twistingmemoirs.com/index.php/ujrra/article/view/190>.

SALMAN, H. A.; KALAKECH, A.; STEITI, A. Random forest algorithm overview. **Babylonian Journal of Machine Learning**, Mesopotamian Academic Press, v. 2024, p. 69–79, jun. 2024.

SAMUEL, A. L. Some studies in machine learning using the game of checkers. **IBM J. Res. Dev.**, IBM Corp., USA, v. 3, n. 3, p. 210–229, jul. 1959. ISSN 0018-8646. Disponível em: <https://doi.org/10.1147/rd.33.0210>.

SANDER, J. et al. Density-based clustering in spatial databases: The algorithm GDBSCAN and its applications. **Data Mining and Knowledge Discovery**, v. 2, n. 2, p. 169–194, 1998.

SCHUBERT, E. et al. DBSCAN revisited, revisited. **ACM Trans. Database Syst.**, Association for Computing Machinery (ACM), v. 42, n. 3, p. 1–21, set. 2017.

SCIKIT-LEARN DEVELOPERS. **sklearn.ensemble.RandomForestClassifier**. 2026. Documentação da biblioteca scikit-learn. Disponível em: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>. Acesso em: 23 out. 2026.

SHALEV-SHWARTZ, S.; BEN-DAVID, S. **Understanding machine learning**. Cambridge, England: Cambridge University Press, 2014.

SHAMS, M. Y. et al. Water quality prediction using machine learning models based on grid search method. **Multimed. Tools Appl.**, Springer Science and Business Media LLC, v. 83, n. 12, p. 35307–35334, set. 2023.

SOLORIO-FERNÁNDEZ, S.; CARRASCO-OCHOA, J. A.; MARTÍNEZ-TRINIDAD, J. F. A review of unsupervised feature selection methods. **Artificial Intelligence Review**, v. 53, n. 2, p. 907–948, 2020. ISSN 1573-7462. Disponível em: <https://doi.org/10.1007/s10462-019-09682-y>.

STONE, M. Cross-validation and multinomial prediction. **Biometrika**, JSTOR, p. 509–515, 1974.

SULAIMAN, R. B.; SCHETININ, V.; SANT, P. Review of machine learning approach on credit card fraud detection. **Human-Centric Intelligent Systems**, Springer, v. 2, p. 55–68, 2022.

VARMA, S.; SIMON, R. Bias in error estimation when using cross-validation for model selection. **BMC bioinformatics**, Springer, v. 7, n. 1, p. 91, 2006.

WADE, C. **Hands-On Gradient Boosting with XGBoost and scikit-learn**. Birmingham, England: Packt Publishing, 2020.

WEST, J.; BHATTACHARYA, M. Intelligent financial fraud detection: A comprehensive review. **Comput. Secur.**, Elsevier BV, v. 57, p. 47–66, mar. 2016.

XGBOOST DEVELOPERS. **XGBoost Parameters**. 2026. Documentação da biblioteca XGBoost. Disponível em: <https://xgboost.readthedocs.io/en/stable/parameter.html>. Acesso em: 23 out. 2026.

ZHANG, E.; ZHANG, Y. Average precision. In: LIU, L.; ÖZSU, M. T. (Ed.). **Encyclopedia of Database Systems**. Boston, MA: Springer, 2009. p. 192–193.

ZHANG, H.; SINGER, B. H. **Recursive partitioning and applications**. 2. ed. New York, NY: Springer, 2010. (Springer Series in Statistics).